

Final Project Report

TLS Fingerprinting for Malicious vs. Benign Traffic Classification

Mahesh Babu Tasil (mbtasil)
Vijaysimha Bandaru (vijaysb)
Sai Soma (somasai)

1 Project Overview

This project investigates the classification of encrypted network traffic using metadata-based machine learning methods. Since a large portion of modern internet communication is encrypted, traditional intrusion detection systems that depend mainly on packet payload inspection are less effective. Decrypting traffic can also raise privacy, legal, and computational concerns. Therefore, this project focuses on using observable traffic metadata instead of encrypted payload contents.

The main goal of this project is to design and evaluate a machine learning system that can classify network traffic as benign or malicious using metadata-based features. The main malicious versus benign experiment uses the Kaggle Network Intrusion Dataset, which contains CICIDS/ISCX-style flow-level records with benign and attack labels. The models used in this project are Decision Tree, Random Forest, and XGBoost.

Because the original project topic is motivated by TLS fingerprinting, an additional CESNET-TLS22 experiment was included. CESNET-TLS22 provides real TLS traffic metadata and application/service labels. In the loaded subset used for this project, the available features included TLS traffic metadata and flow-level packet features such as bytes, packets, duration, and packet-position information. Since the CESNET-TLS22 subset used here does not provide verified malicious and benign labels, it is reported as a TLS traffic metadata proof-of-concept rather than direct malicious traffic detection.

This project also includes a synthetic perturbation stress test. Random noise was added to the test features to evaluate how sensitive the trained model is when the feature distribution changes. This is not the same as real-world concept drift, but it gives an initial understanding of model robustness under changed or corrupted feature conditions.

2 Motivation and Relevance

Encrypted traffic analysis is an important problem in modern cybersecurity. Encryption improves user privacy and protects sensitive data, but it also allows attackers to hide malicious activity inside encrypted communication channels. Examples include malware command-and-control communication, data exfiltration, botnet activity, phishing infrastructure, and distributed denial-of-service traffic.

Traditional security tools often rely on deep packet inspection or payload-based signatures. However, these methods are limited when traffic is encrypted. TLS fingerprinting and metadata-based traffic analysis offer an alternative approach. Instead of decrypting traffic, the system analyzes observable characteristics such as packet counts, byte counts, flow duration, packet length behavior, packet timing, and other statistical features.

This project is relevant because it supports a privacy-preserving approach to suspicious traffic detection. It allows security analysts to identify suspicious encrypted or network traffic behavior without directly inspecting user payload data. This can be useful in enterprise security monitoring, intrusion detection systems, and security operations center workflows.

3 Project Implementation

3.1 Literature Review

During the project, we reviewed concepts related to encrypted traffic classification, TLS fingerprinting, JA3-style fingerprinting, metadata-based intrusion detection, and machine learning for network security. Prior work suggests that encrypted traffic can still leave observable patterns through handshake metadata, flow behavior, packet sizes, timing, and statistical features.

Although encrypted payloads are not visible, metadata can still provide useful signals for classification. For example, malicious traffic may show unusual flow duration, abnormal packet length patterns, unexpected byte ratios, or different communication behavior compared with benign traffic. Based on this idea, this project uses machine learning to learn patterns from network traffic metadata.

3.2 Dataset Description

This project uses two datasets for two related purposes.

3.2.1 Kaggle Network Intrusion Dataset

The main malicious versus benign classification experiment uses the Kaggle Network Intrusion Dataset, which provides CICIDS/ISCX-style network traffic CSV files. The following files were used:

- Monday-WorkingHours.pcap_ISCX.csv
- Tuesday-WorkingHours.pcap_ISCX.csv
- Wednesday-workingHours.pcap_ISCX.csv
- Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv

Each CSV file contains flow-level network traffic records. The files include benign traffic as well as attack traffic. The original dataset labels were converted into a binary classification problem. The label `BENIGN` was assigned class 0, and all other attack labels were assigned class 1. Therefore, the final task for this dataset is binary classification:

- Class 0: Benign traffic
- Class 1: Malicious traffic

After combining the four CSV files, the dataset contained 1,894,275 total records and 79 columns. After removing infinite values and missing values, the cleaned dataset contained 1,892,243 records. The final binary label distribution contained 1,498,663 benign samples and 393,580 malicious samples.

Before removing selected leakage-prone features, the dataset contained 78 numeric input features. After removing selected leakage-prone columns, 68 numeric features were used for model training. The dataset was split into 70% training and 30% testing subsets using stratification. The training set contained 1,324,570 samples, and the testing set contained 567,673 samples.

A check for TLS-specific fields was performed on this dataset. No explicit TLS-specific fields such as `TLS_JA3`, `TLS_SNI`, cipher suites, TLS extensions, ALPN, or certificate metadata were found. Therefore, the Kaggle experiment is treated as flow-level metadata-based malicious versus benign classification.

3.2.2 CESNET-TLS22 Dataset

To better align the project with TLS fingerprinting, an additional experiment was performed using CESNET-TLS22 through the CESNET DataZoo package. CESNET-TLS22 is a real TLS traffic dataset. In this project, the XS version was loaded to keep the experiment manageable in Google Colab.

The loaded CESNET-TLS22 subset used 30,000 training samples and 15,000 testing samples. The available columns in the DataZoo-loaded version were:

- PPI
- BYTES

- BYTES_REV
- PACKETS
- PACKETS_REV
- DURATION
- PPI_LEN
- PPI_ROUNDTRIPS
- PPI_DURATION
- APP

For the CESNET experiment, the following eight metadata features were selected:

- BYTES
- BYTES_REV
- PACKETS
- PACKETS_REV
- DURATION
- PPI_LEN
- PPI_ROUNDTRIPS
- PPI_DURATION

The `APP` column was used as the service/application label. Since CESNET-TLS22 does not provide verified malicious and benign labels, the CESNET experiment was not treated as direct malicious traffic detection. Instead, it was used as a TLS traffic metadata proof-of-concept. The majority APP class was treated as class 0, and all other APP classes were grouped as class 1.

For the sampled CESNET-TLS22 subset, APP 124 was the majority APP class. The training distribution contained 1,027 samples for the majority APP class and 28,973 samples for other APPs. The testing distribution contained 526 samples for the majority APP class and 14,474 samples for other APPs.

3.3 Labeling Process

For the Kaggle Network Intrusion Dataset, the original dataset contains a `Label` column. This column was cleaned by removing extra spaces and converting values into a consistent format. After cleaning, the labels were converted into binary values. The conversion rule was:

$$\text{label} = \begin{cases} 0, & \text{if Label is BENIGN} \\ 1, & \text{otherwise} \end{cases}$$

The combined Kaggle dataset included multiple types of malicious labels, including DDoS, DoS Hulk, DoS GoldenEye, DoS slowloris, DoS Slowhttptest, FTP-Patator, SSH-Patator, and Heartbleed. These were grouped into the malicious class for binary classification.

For the CESNET-TLS22 experiment, the dataset did not provide malicious and benign ground-truth labels. Therefore, a binary proof-of-concept label was created from the `APP` service label. The majority APP was treated as class 0, and all other APP values were treated as class 1. This was done only to demonstrate that the same tree-based machine learning pipeline can learn from real TLS traffic metadata. It should not be interpreted as verified malicious versus benign classification.

3.4 Data Preprocessing and Feature Engineering

For the Kaggle dataset, preprocessing included loading and combining CSV files, cleaning column names, converting the label column into a binary target, replacing infinite values with missing values, removing missing values, and selecting numeric features.

Some features were removed because they could cause overly optimistic results or possible information leakage. Examples of removed columns include:

- Flow Bytes/s
- Flow Packets/s
- Total Fwd Packets
- Total Backward Packets
- Total Length of Fwd Packets
- Total Length of Bwd Packets
- Packet Length Mean
- Packet Length Std
- Packet Length Variance

- Average Packet Size

The preprocessing workflow was implemented carefully to reduce leakage risk. The dataset was split into training and testing sets first. Then, `StandardScaler` was fitted only on the training features and applied to both training and testing features. This prevents the scaler from learning information from the test set.

For the CESNET-TLS22 experiment, the sampled train and test sets were loaded using CESNET DataZoo. Since the selected models were tree-based models, feature scaling was not required for the CESNET experiment. The selected fields were cleaned and used directly for model training.

3.5 Model Implementation

Three tree-based machine learning models were implemented:

1. Decision Tree
2. Random Forest
3. XGBoost

The Decision Tree model was used as a baseline model. Random Forest was used as an ensemble model to improve stability and reduce overfitting compared with a single tree. XGBoost was used as a boosted ensemble method because it often performs well on structured tabular datasets.

The implementation was completed in Python using Google Colab. The main libraries used were Pandas, NumPy, Scikit-learn, XGBoost, Matplotlib, Seaborn, and CESNET DataZoo. The evaluation metrics used were accuracy, F1-score, ROC-AUC, classification report, confusion matrix, ROC curve, and feature importance analysis.

4 Final Results and Evaluation

4.1 Kaggle Network Intrusion Dataset Results

The models were trained on the cleaned and standardized Kaggle dataset. The test set was used to evaluate model performance. The evaluation focused on accuracy, F1-score, and ROC-AUC.

Table 1: Kaggle/CICIDS Malicious vs. Benign Model Performance

Model	Accuracy	F1-Score	ROC-AUC
Decision Tree	0.9922	0.9811	0.9949
Random Forest	0.9997	0.9993	1.0000
XGBoost	0.9997	0.9993	1.0000

The Decision Tree baseline achieved an accuracy of 0.9922, F1-score of 0.9811, and ROC-AUC of 0.9949. Random Forest and XGBoost both achieved very high performance, with accuracy of 0.9997, F1-score of 0.9993, and ROC-AUC of 1.0000. XGBoost was selected as one of the final models because it provided strong performance and is well suited for structured tabular network traffic data.

However, the near-perfect performance should be interpreted carefully. Even though preprocessing leakage was reduced by fitting the scaler only on the training data, public intrusion detection datasets may contain repeated or highly similar flows. A random train-test split may allow similar flows from the same capture period to appear in both training and testing sets. Therefore, the high performance should not be interpreted as guaranteed real-world deployment performance.

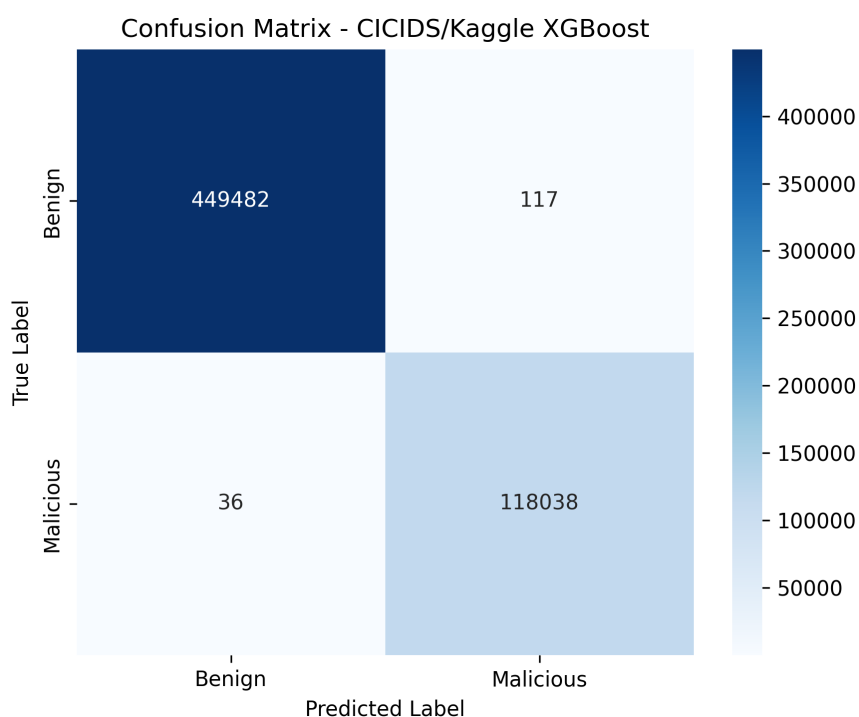


Figure 1: Confusion Matrix for Kaggle/CICIDS XGBoost Model

The XGBoost confusion matrix showed that the model correctly classified 449,482 benign samples as benign and 118,038 malicious samples as malicious. It produced 117 false positives, where benign traffic was incorrectly flagged as malicious, and 36 false negatives, where malicious traffic was missed. From a cybersecurity perspective, the low number of false negatives is important because missed malicious traffic can create security risk. The low number of false positives also means the model would create fewer unnecessary alerts for analysts.

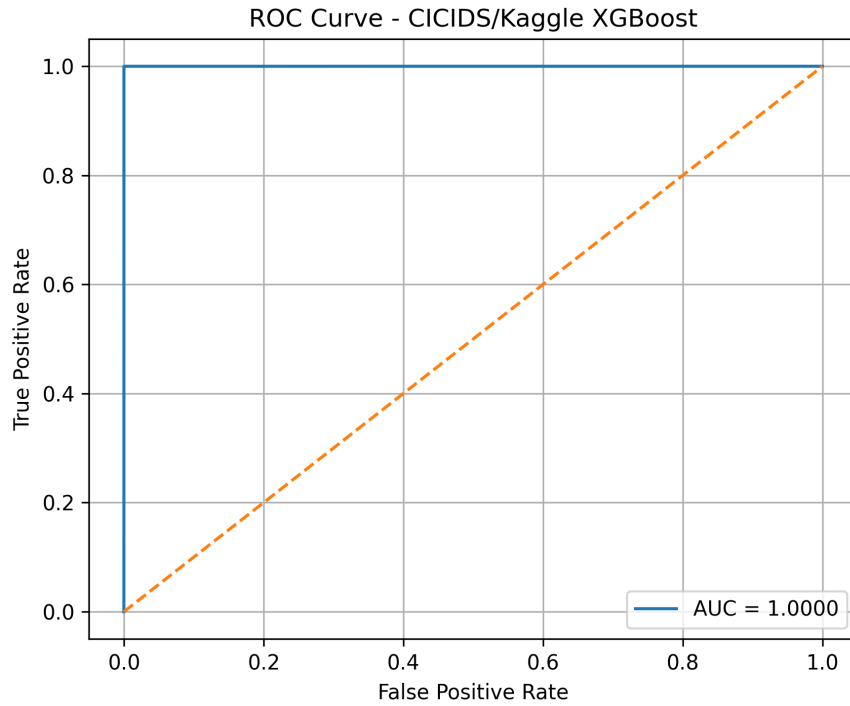


Figure 2: ROC Curve for Kaggle/CICIDS XGBoost Model

The ROC curve shows the relationship between true positive rate and false positive rate. A higher ROC-AUC value indicates better separation between benign and malicious traffic.

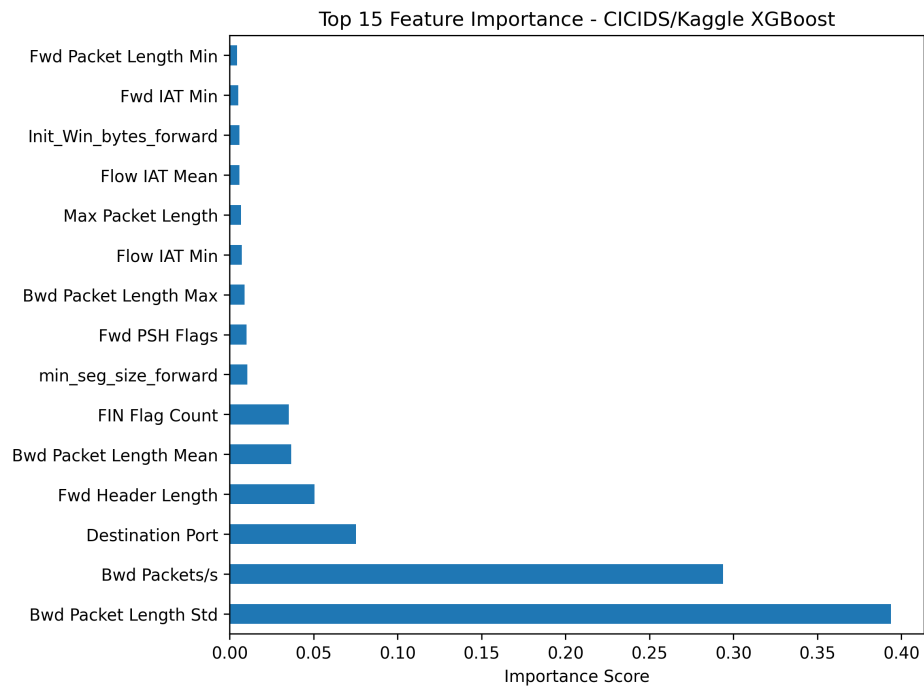


Figure 3: Top Feature Importance Values for Kaggle/CICIDS XGBoost

Feature importance analysis was used to understand which features contributed most to the final model prediction. This helps make the model more interpretable and provides insight into which traffic characteristics are useful for identifying malicious behavior.

4.2 Synthetic Perturbation Stress Test

A synthetic perturbation stress test was performed to evaluate how the final model behaves when the test feature distribution is artificially changed. In this experiment, random noise was added to the standardized test feature values. This does not fully represent real-world concept drift, but it provides an initial robustness check by showing how sensitive the model is to feature perturbation or corrupted measurements.

Table 2: XGBoost Performance Before and After Synthetic Perturbation

Condition	Accuracy	F1-Score	ROC-AUC
Original Test Data	0.9997	0.9993	1.0000
Perturbed Test Data	0.7576	0.0524	0.6492

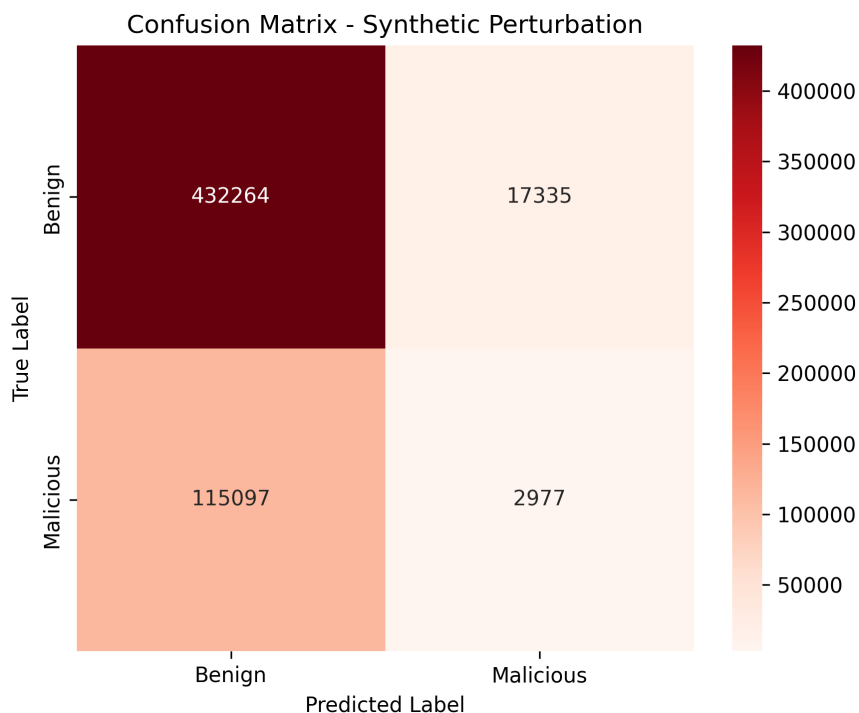


Figure 4: Confusion Matrix Under Synthetic Perturbation

Under the synthetic perturbation condition, the model performance dropped significantly. The model correctly classified 432,264 benign samples, but it incorrectly flagged 17,335 benign sam-

ples as malicious. More importantly, it missed 115,097 malicious samples and correctly detected only 2,977 malicious samples. This shows that the model became much weaker when the test feature distribution was changed. The large increase in false negatives is a serious concern because many malicious flows would not be detected under shifted or corrupted feature conditions.

4.3 CESNET-TLS22 TLS Traffic Metadata Proof-of-Concept Results

CESNET-TLS22 was added as a real TLS traffic metadata proof-of-concept experiment. The selected CESNET features were BYTES, BYTES_REV, PACKETS, PACKETS_REV, DURATION, PPI_LEN, PPI_ROUNDTRIPS, and PPI_DURATION. The APP label was converted into a binary proof-of-concept label, where the majority APP was class 0 and all other APPs were class 1.

Table 3: CESNET-TLS22 TLS Traffic Metadata Proof-of-Concept Results

Model	Accuracy	F1-Score	ROC-AUC
Decision Tree	0.9694	0.9844	0.7903
Random Forest	0.9762	0.9878	0.9032
XGBoost	0.9733	0.9863	0.8897

The CESNET-TLS22 proof-of-concept results show that tree-based models can learn useful patterns from TLS traffic metadata. Random Forest achieved the strongest ROC-AUC score of 0.9032, while XGBoost also performed strongly with an accuracy of 0.9733 and F1-score of 0.9863. However, because the binary label was created from the APP service label rather than true malicious and benign ground truth, these results should be interpreted only as a TLS traffic metadata classification proof-of-concept.

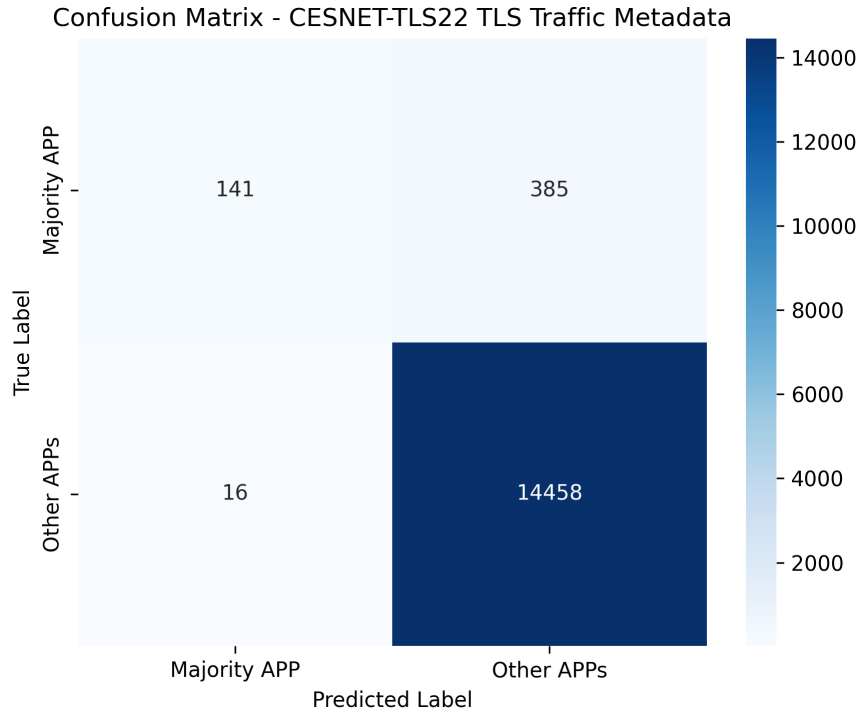


Figure 5: Confusion Matrix for CESNET-TLS22 TLS Traffic Metadata Experiment

The CESNET-TLS22 confusion matrix shows that the model correctly classified 141 majority-APP samples and 14,458 other-APP samples. It misclassified 385 majority-APP samples as other APPs and 16 other-APP samples as the majority APP. This result reflects the imbalance of the proof-of-concept task, where most samples belong to the other-APP group.

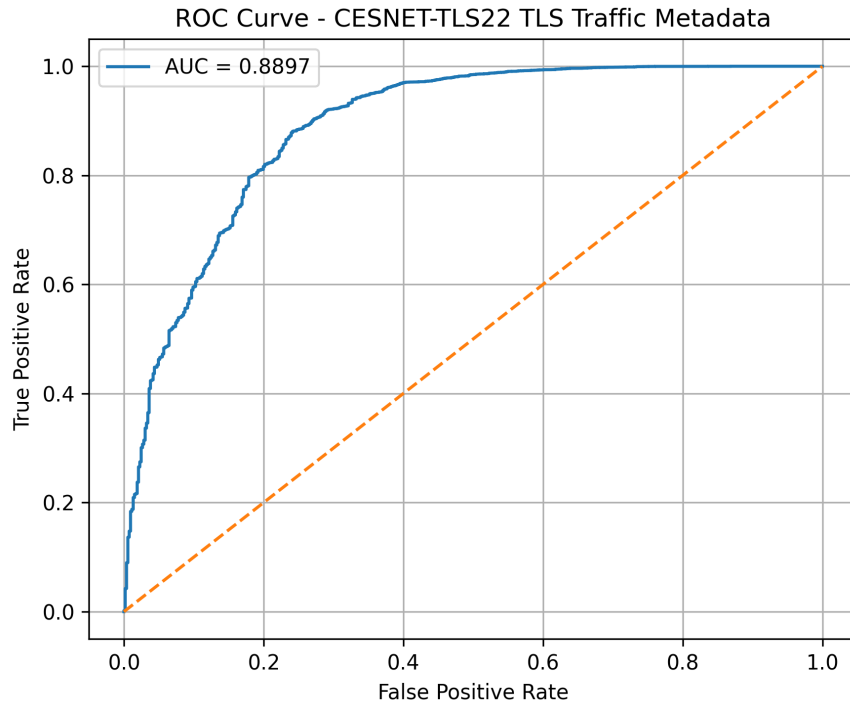


Figure 6: ROC Curve for CESNET-TLS22 TLS Traffic Metadata Experiment

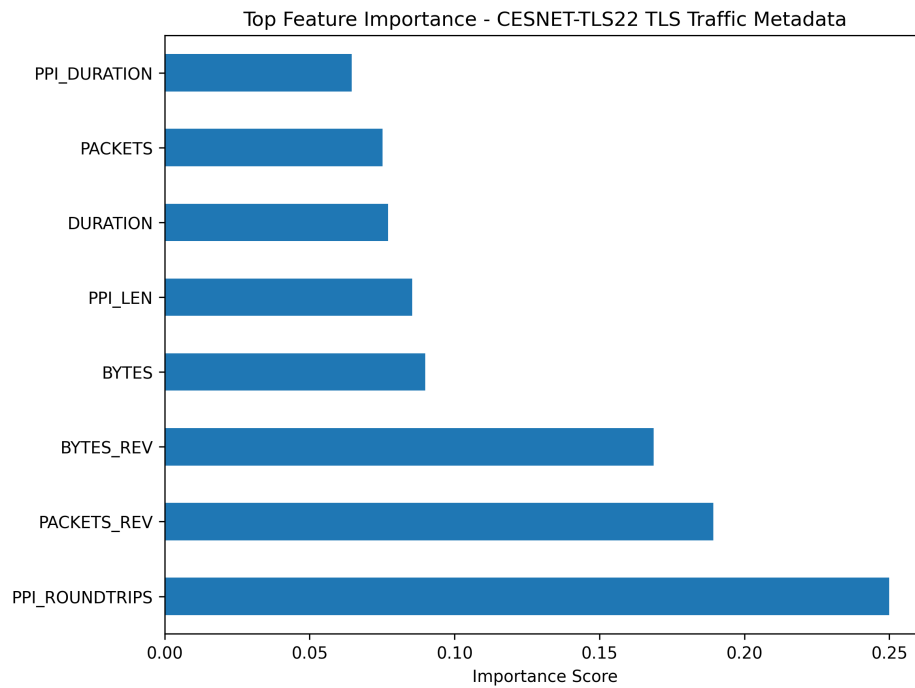


Figure 7: Top Feature Importance Values for CESNET-TLS22 TLS Traffic Metadata Experiment

5 Challenges and Limitations

Several challenges and limitations were identified during this project.

First, the main Kaggle dataset is based on public CICIDS/ISCX-style traffic. Although this dataset is useful for experimentation, it may not fully represent modern enterprise traffic. Real-world encrypted traffic can be more diverse, noisy, and dynamic.

Second, the main malicious versus benign Kaggle experiment uses flow-level metadata instead of full TLS handshake fingerprints. This is useful for privacy-preserving detection, but it does not capture all details available in complete TLS fingerprinting approaches such as JA3, JA3S, cipher suite sequences, TLS extensions, SNI, ALPN, and certificate metadata.

Third, even after removing some leakage-prone features and fitting the scaler only on the training data, there is still a possibility that dataset-specific patterns or similar flows across the random split make classification easier than it would be in a real deployment. Therefore, high performance should be interpreted carefully.

Fourth, the synthetic perturbation stress test was simulated by adding random noise to the test features. This provides a useful initial understanding of model sensitivity, but it is not the same as evaluating the model on real traffic collected across different time periods.

Fifth, the CESNET-TLS22 experiment does not use verified malicious and benign labels. The binary target was created from the APP service label, where the majority APP was treated as class 0 and all other APPs were grouped as class 1. Therefore, this experiment is reported only as a TLS traffic metadata proof-of-concept.

Finally, this project focuses mainly on binary classification and proof-of-concept evaluation. A real security system may also need multi-class attack classification, severity scoring, model explainability, alert prioritization, and integration with analyst workflows.

6 Final Project Outcomes

The final project successfully implemented a machine learning pipeline for malicious versus benign traffic classification using metadata-based network traffic features. The project also added a TLS traffic metadata proof-of-concept experiment using CESNET-TLS22.

The completed work includes:

- Loading and combining multiple public network traffic CSV files from the Kaggle Network Intrusion Dataset.
- Cleaning labels and converting the Kaggle problem into binary malicious versus benign classification.
- Handling missing and infinite values.
- Removing selected leakage-prone features.

- Fitting the scaler only on the training data to reduce preprocessing leakage risk.
- Training Decision Tree, Random Forest, and XGBoost models.
- Evaluating models using accuracy, F1-score, ROC-AUC, classification reports, confusion matrices, and ROC curves.
- Performing feature importance analysis.
- Conducting a synthetic perturbation stress test.
- Adding CESNET-TLS22 as a real TLS traffic metadata proof-of-concept experiment.
- Preparing a GitHub repository with code, outputs, report, requirements file, and dataset access instructions.

The project demonstrates that metadata-based traffic analysis can be useful for detecting malicious traffic without decrypting payloads. It also shows that real TLS traffic metadata can be used in a tree-based classification pipeline. However, the results also show that model reliability depends on dataset quality, feature selection, label quality, and robustness against distribution changes.

7 Report Quality and Organization

This final report follows the same overall structure as the midterm project template. It includes the project overview, motivation, implementation details, final results, limitations, outcomes, and future work. The report is organized to clearly explain the project from problem definition to final evaluation.

The GitHub repository contains the materials produced and used in the project, including the notebook, Python script, output figures, final report, requirements file, and README file. The README file explains how to access the Kaggle Network Intrusion Dataset and how to use the CESNET DataZoo package for the CESNET-TLS22 experiment. Since the dataset files are large, they are not directly included in the repository. Instead, the README provides instructions for downloading or loading the datasets.

8 Future Work

Future work for this project includes improving the reliability and real-world applicability of the TLS fingerprinting detection system. One important direction is to evaluate the models on larger

and more diverse encrypted traffic datasets. The current work demonstrates that flow-level metadata and TLS traffic metadata can be useful for classification, but additional datasets are needed to confirm that the model generalizes beyond the selected experimental data.

Another future improvement is to include complete TLS fingerprinting features, such as full JA3 and JA3S fingerprints, detailed cipher suite sequences, TLS extensions, SNI, ALPN, certificate metadata, and temporal behavior of network flows. These features may improve detection performance and provide stronger behavioral indicators of malicious encrypted traffic.

Real concept drift should also be studied more deeply. The current stress test used synthetic noise, but a stronger drift evaluation would use time-based or file-based splits. For example, future work can train the model on earlier traffic periods and test it on later traffic periods. This would better represent changes in attack behavior, protocol usage, and network traffic patterns over time.

The project can also be extended into a practical security monitoring pipeline. The trained model could be connected with tools such as Wireshark, Zeek, tshark, or CICFlowMeter to extract TLS and flow metadata from live traffic. This would allow the system to support real-time alerting and help security analysts identify suspicious encrypted connections without decrypting packet contents.

Finally, future versions of this project can include model explainability methods such as SHAP or LIME. These methods would help security analysts understand why a traffic flow was classified as malicious. This is important because security tools should not only produce alerts, but also provide useful explanations that support investigation and response.

References

1. Kaggle. Network Intrusion Dataset. Available at: <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset?resource=download>
2. CESNET. CESNET-TLS22 Dataset and CESNET DataZoo. Available at: <https://cesnet.github.io/cesnet-datazoo/>
3. Scikit-learn Developers. Scikit-learn: Machine Learning in Python. Available at: <https://scikit-learn.org/>
4. XGBoost Developers. XGBoost Documentation. Available at: <https://xgboost.readthedocs.io/>
5. Salesforce Engineering. JA3: SSL/TLS Client Fingerprinting for Malware Detection. Available at: <https://github.com/salesforce/ja3>
6. Wireshark Foundation. Wireshark Network Protocol Analyzer. Available at: <https://www.wireshark.org/>
7. Zeek Project. Zeek Network Security Monitor. Available at: <https://zeek.org/>

GitHub Repository

The complete project materials, including source code, notebook, output figures, report, and dataset access instructions, are available in the GitHub repository below:

GitHub Link:

<https://github.com/vijaysimha-13/CPRE-5600-Final-Project.git>